SAIL VOCATIONAL TRAINING PROJECT REPORT ON

CSV COMPARISON TOOL

By

Ankit Kumar

B.TechinComputerScienceEngineering

IILMUNIVERSITY, GREATER NOIDA

Submitted to

HRD,SAIL

Guidedby

Mr. Chandan Kumar
Assistant General Manager
(AGM), C & IT
Steel Authority of India, Bokaro Steel City

STEEL AUTHORITY OF INDIA LIMITED

INDEX

CERTIFICATE

This is to certify that the project work entitled "**CSV COMPARISON TOOL**" is done by Ankit Kumar.
URN NO: 5913893 in partial fulfillment of the requirements for the Vocational Training (VT) in **Computer & IT** for 4weeks (02-06-2025 to 28-06-2025).
This work is submitted to the department as a part of evaluation of project.

SIGNATURE **DATE**

ACKNOWLEDGEMENT

I am deeply grateful to all those who supported and guided me throughout the development of this project titled "CSV Comparator Tool for Employee Data Management" at the steel plant.

First and foremost, I would like to express my heartfelt thanks to the Department of Human Resources and IT, whose practical insights into employee data handling provided the foundation for identifying the real-world need for this tool. Their cooperation in sharing the master and updated datasets was instrumental in the success of this project.

I extend my sincere appreciation to my project guide and mentors, whose guidance, technical advice, and valuable feedback greatly enhanced the functionality and usability of the tool.

A special note of thanks goes to the data management and payroll teams for their continuous support in testing the tool with live datasets involving over 932 employee records, which helped validate the accuracy and robustness of the comparison logic.

Lastly, I would like to acknowledge the open-source Python community and the developers of libraries like tkinter, pandas, and csv, without which this project would not have been possible.

This project has not only enhanced my technical skills but also provided valuable experience in solving practical challenges within an industrial data environment.

INTRODUCTION

The steel industry playsa crucial role inthe economic development of any nation. It is often referred to as the backbone of modern industrial society due to its widespread applications across infrastructure, construction, transportation, manufacturing, and defense sectors.

Steel is an alloy primarily composed of iron and carbon and is valued for its high strength, durability, and versatility. The industry includes a wide range of processes—from mining and refining raw materials, to producing finished steel products in integrated steel plants or mini-mills.

Global Significance:

- The steel industry is one of the largest and most essential global industries.
- Countries like China, India, Japan, the U.S., and Russia are among the top steel producers.
- The demand for steel continues to grow with urbanization, industrialization, and infrastructural development.

Steel Industry in India:

- India is currently the second-largest producer of crude steel in the world.
- The Indian steel sector is dominated by public sector undertakings (PSUs) such as SAIL (Steel Authority of India Limited) and private players like Tata Steel and JSW Steel.
- The industry supports over 2 million jobs directly and millions more indirectly through associated industries.

Importance of Steel:

- Construction: Bridges, buildings, pipelines, railways, and roads.
- Automotive: Manufacturing of cars, trucks, and heavy vehicles.
- Defense and Aerospace: High-grade steels are used in defense equipment and aircraft.
- Energy: Used in the production and transport of oil, gas, and electricity.

Current Trends:

- Emphasis on green steel and environmental sustainability.
- Adoption of Industry 4.0 technologies like automation, data analytics, and machine learning.
- Focus on increasing energy efficiency and reducing carbon emission

BOKAROSTEELPLANT (SAIL BSL)

The Bokaro Steel Plant (BSL) is the fourth integrated steel plant in the public sector, established with assistance from the Soviet Union. It was initially incorporated as a limited company on 29 January 1964 and later merged with the Steel Authority of India Limited (SAIL)—first as a subsidiary and subsequently as a unit—under the Restructuring & Miscellaneous Provisions Act, 1978.

Construction officially commenced on 6 April 1968. The plant currently operates five blast furnaces with a liquid steel production capacity of **5.25 million tonnes (MT)**. A major **brownfield expansion project worth ₹20,000 crore** is underway to enhance capacity to **7.55 MT by 2025**. Continuous modernization and sustainability initiatives are being implemented, including decarbonization and renewable energy projects.

During recent years, BSL has seen record-breaking production and technological improvements. In FY 2023–24, BSL achieved its highest-ever hot metal production of **4.73 MT** and crude steel output of **4.31 MT**. The plant also demonstrated the best-ever techno-economic parameters such as blast furnace productivity and energy efficiency. Key upgrades include new slab casting units and advanced automation, while BSL continues collaboration with global players for technology and process improvement.

For the establishment of the plant, approximately 64 moujas (land units, each potentially including multiple villages) were acquired. Of the total land acquired, 7,765 hectares were utilized for the steel plant infrastructure. Some parts of the land were later allocated by SAIL to private entities, allegedly without formal government approval.

In terms of financial performance:

- In FY 2020–21, the plant recorded a **Profit Before Tax (PBT) of ₹2,251 crore**, a massive rise compared to ₹48 crore in the previous year.
- In FY 2021–22, BSL achieved a record **PBT of ₹6,387 crore**, the highest among all SAIL units.
- In FY 2022–23, BSL contributed a net profit of approximately **₹600 crore** to SAIL's total profit.
- In FY 2023–24, SAIL reported a consolidated **PAT of ₹3,688 crore**, with BSL playing a key role in achieving record steel output.

The plant continues to focus on energy transition with a 100 MW renewable energy target and aims to reduce CO_2 emissions from **2.67 to below 2.2 tonnes per tonne of crude steel by 2030**. It remains a crucial pillar in India's steel manufacturing ecosystem.

In the financial year 2020–21, under the leadership of Managing Director Mr. Amarendu Prakash, the Bokaro Steel Plant surpassed its production and financial targets. It reported a Profit Before Tax (PBT) of ₹2,251 crore, a remarkable rise compared to the ₹48 crore PBT during the corresponding period last year (CPLY).

OVERVIEW ROLE OF IT IN STEEL INDUSTRY

The Information Technology (IT) sector has become a transformative force in the steel industry, reshaping traditional manufacturing processes, improving efficiency, reducing costs, and enhancing decision-making. In an era of digital transformation and Industry 4.0, IT acts as a critical enabler of innovation in steel production, logistics, quality control, and workforce management.

1. Automation and Process Control

 Supervisory Control and Data Acquisition (SCADA) and Distributed Control Systems (DCS) automate and monitor critical operations like blast furnace temperatures, rolling mill speeds, and cooling systems.

 These systems ensure real-time process optimization, reduce human errors, and improve safety and reliability.

2. Enterprise Resource Planning (ERP) Systems

 Steel plants like SAIL, Tata Steel, and JSW rely on ERP platforms (e.g., SAP, Oracle) for integrated management of supply chain, inventory, procurement, maintenance, finance, and HR.

 ERP streamlines business operations by providing centralized access to real-time data and performance metrics.

3. Predictive Maintenance

 IT enables condition monitoring of machines and equipment using IoT sensors and AI algorithms.

 Predictive maintenance minimizes unplanned downtime, extends asset lifespan, and reduces operational costs.

4. Quality Assurance and Data Analytics

 Computer Vision and AI-based systems are used to inspect steel surfaces and detect flaws in real time.

 Data analytics helps in analyzing product quality, performance trends, and customer feedback to drive continuous improvement.

5. Supply Chain and Logistics Optimization

 IT tools are used for supply chain visibility, fleet tracking, inventory planning, and order fulfillment.

 Integration of RFID, GPS, and warehouse management systems ensures timely delivery and reduces waste.

ABSTRACT & PROBLEM STATEMENTS

Abstract:
The CSV Comparator is a desktop application developed using Python's Tkinter library, designed to compare two CSV files—typically referred to as the master and slave datasets. The tool automatically identifies and displays the differences between the two files, highlighting updated fields, newly added records, and other discrepancies. With a user-friendly graphical interface, the application simplifies the data validation process, making it especially useful for HR, finance, and IT teams that regularly deal with data synchronization, migration, or auditing. The tool supports efficient navigation, real-time searching, and automated report generation in Excel format, ensuring accurate and streamlined comparison results even for large datasets.

Problem Statement:
In many organizational settings, data consistency between source files is crucial for decision-making and operations. However, manually comparing large CSV files to identify updates, new records, or errors is time-consuming, error-prone, and inefficient. Existing spreadsheet tools provide limited support for detailed, field- level comparison and tracking of new entries. There is a clear need for a reliable, automated solution that can:

Compare two CSV files based on unique key columns,

Detect and display differences at the field level,

Identify new entries present in one file but not the other,

Provide a user-friendly interface for browsing and searching results,

Export comparison results to a readable Excel format.

CHALLENGES

1. **File Format Compatibility**
   Handling both .csv and .xlsx files required extra logic and testing to ensure the application could read different formats without errors.

2. **Accurate Field-Level Comparison**
   It was challenging to compare each employee's data column by column and detect only the fields that had actually changed.

3. **Handling Missing or Null Values**
   Comparing values became difficult when one file had missing (NULL) data and the other didn't, which required special conditions in the logic.

4. **Designing a User-Friendly GUI**
   Creating a clean and intuitive interface using Tkinter that beginners could use without confusion was a time-consuming task.

5. **Exporting Data to Excel**
   Providing an option to export the results to Excel and ensuring the file was saved correctly without errors (like file open issues) was tricky to implement.

TECHNOLOGY USED

- **Programming Language:** Python
- **GUI Framework:** Tkinter
- **Data Handling:** pandas,csv
- **File Output:** excel(.xlsxvia pandas)

Employee Comparison Tool (BSL SAIL)

THE CODE

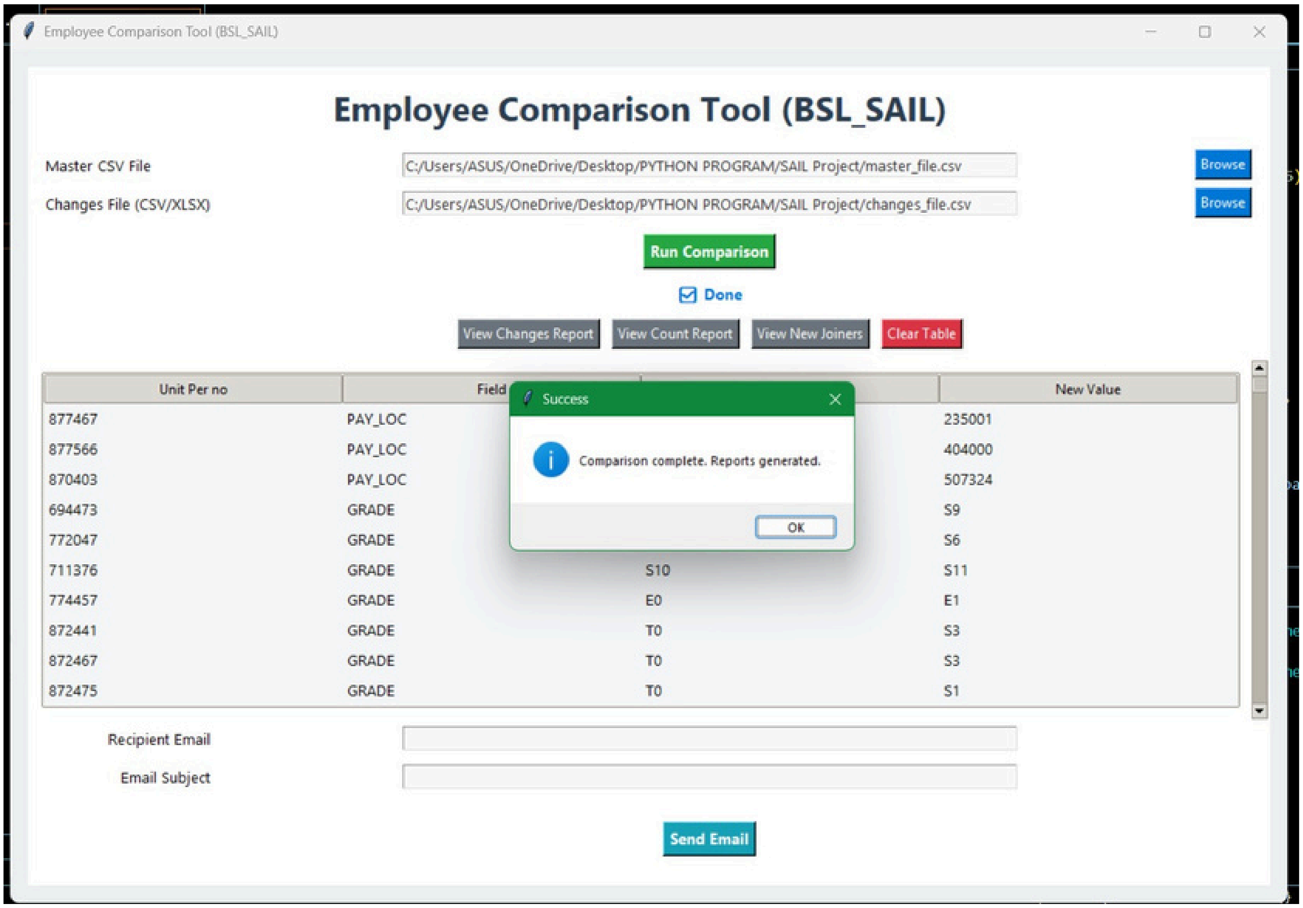
ProjectSAIL.py > ...

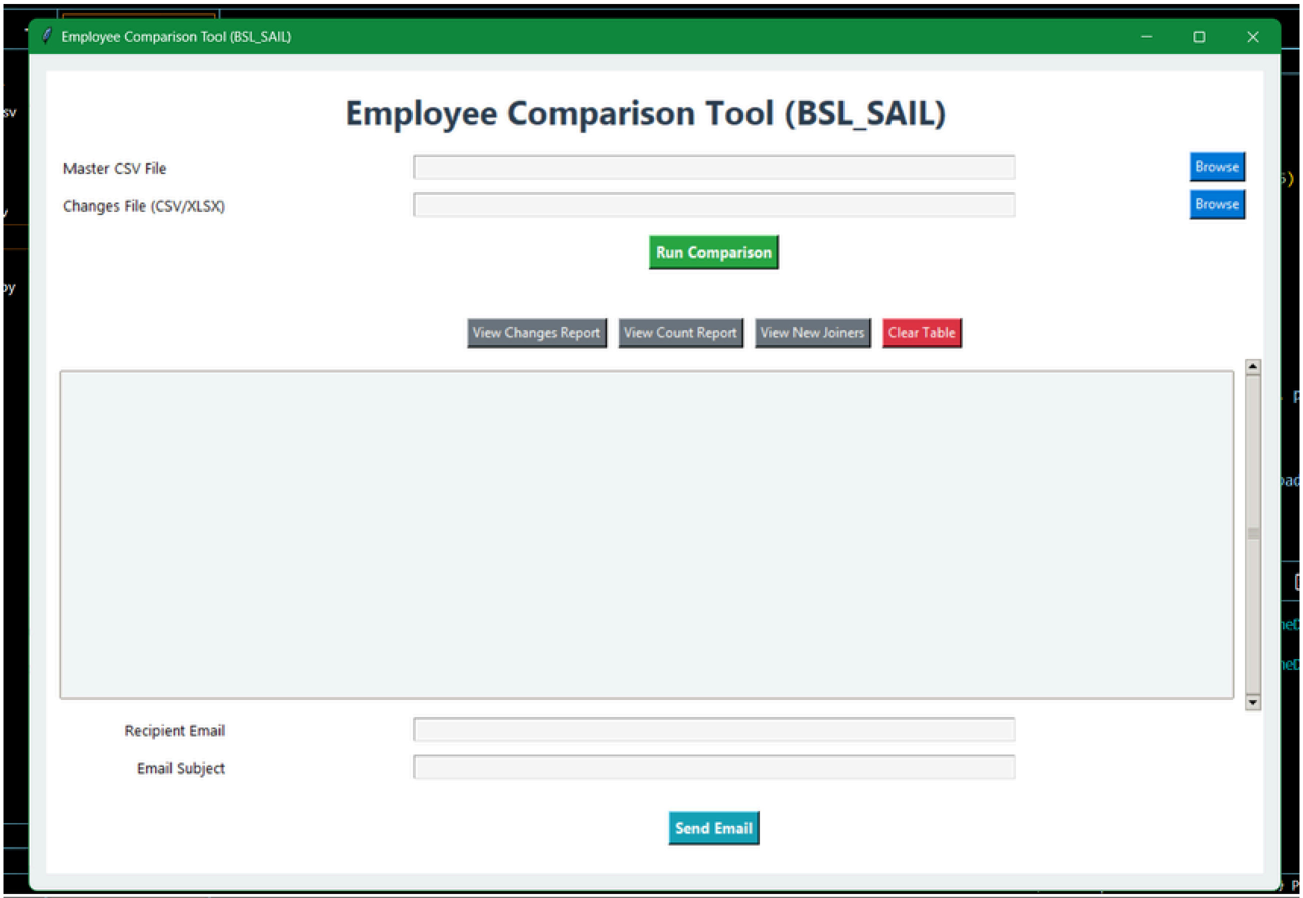
```python
import tkinter as tk
from tkinter import filedialog, messagebox, ttk
import pandas as pd
import os
import smtplib
from email.message import EmailMessage

def apply_styles(widget):
    style = ttk.Style()
    style.theme_use("clam")
    style.configure("Treeview",
                    background="#f4f6f8",
                    foreground="black",
                    rowheight=26,
                    fieldbackground="#f4f6f8",
                    font=("Segoe UI", 10))
    style.map('Treeview', background=[('selected', '#0078D7')])
    widget.configure(bg="#ecf0f1")

def load_file(var, filetypes):
    path = filedialog.askopenfilename(filetypes=filetypes)
    if path:
        var.set(path)

def update_progress(percent_text):
    progress_label.config(text=percent_text)
    root.update_idletasks()

def run_comparison():
    master_path = master_file_path.get()
    changes_path = changes_file_path.get()

    if not os.path.exists(master_path) or not os.path.exists(changes_path):
        messagebox.showerror("Error", "One or both file paths are invalid.")
        return
    try:
        update_progress("10% - Loading files...")
        master_df = pd.read_csv(master_path)
        changes_df = pd.read_excel(changes_path) if changes_path.endswith(".xlsx") else pd.read_csv(changes_path)

        update_progress("25% - Cleaning columns...")
        master_df.columns = master_df.columns.str.upper()
        changes_df.columns = changes_df.columns.str.upper()
        master_df.drop(columns=[col for col in master_df.columns if "YYYYMM" in col], inplace=True, errors='ignore')
        changes_df.drop(columns=[col for col in changes_df.columns if "YYYYMM" in col], inplace=True, errors='ignore')

        update_progress("40% - Merging data...")
        merged_df = pd.merge(master_df, changes_df, on="UNIT_PERNO", suffixes=('_OLD', '_NEW'))

        update_progress("60% - Comparing fields...")
        changes = []
        for col in master_df.columns:
            if col == "UNIT_PERNO":
                continue
            old_col = f"{col}_OLD"
            new_col = f"{col}_NEW"
            if old_col in merged_df.columns and new_col in merged_df.columns:
                changed_rows = merged_df[merged_df[old_col] != merged_df[new_col]]
                for _, row in changed_rows.iterrows():
                    changes.append({
                        "Unit Per no": row["UNIT_PERNO"],
                        "Field": col,
                        "Old Value": row[old_col],
                        "New Value": row[new_col]
                    })

        df_changes = pd.DataFrame(changes)
        df_changes.to_csv("Changes_New.csv", index=False)

        update_progress("75% - Extracting new joinees...")
```

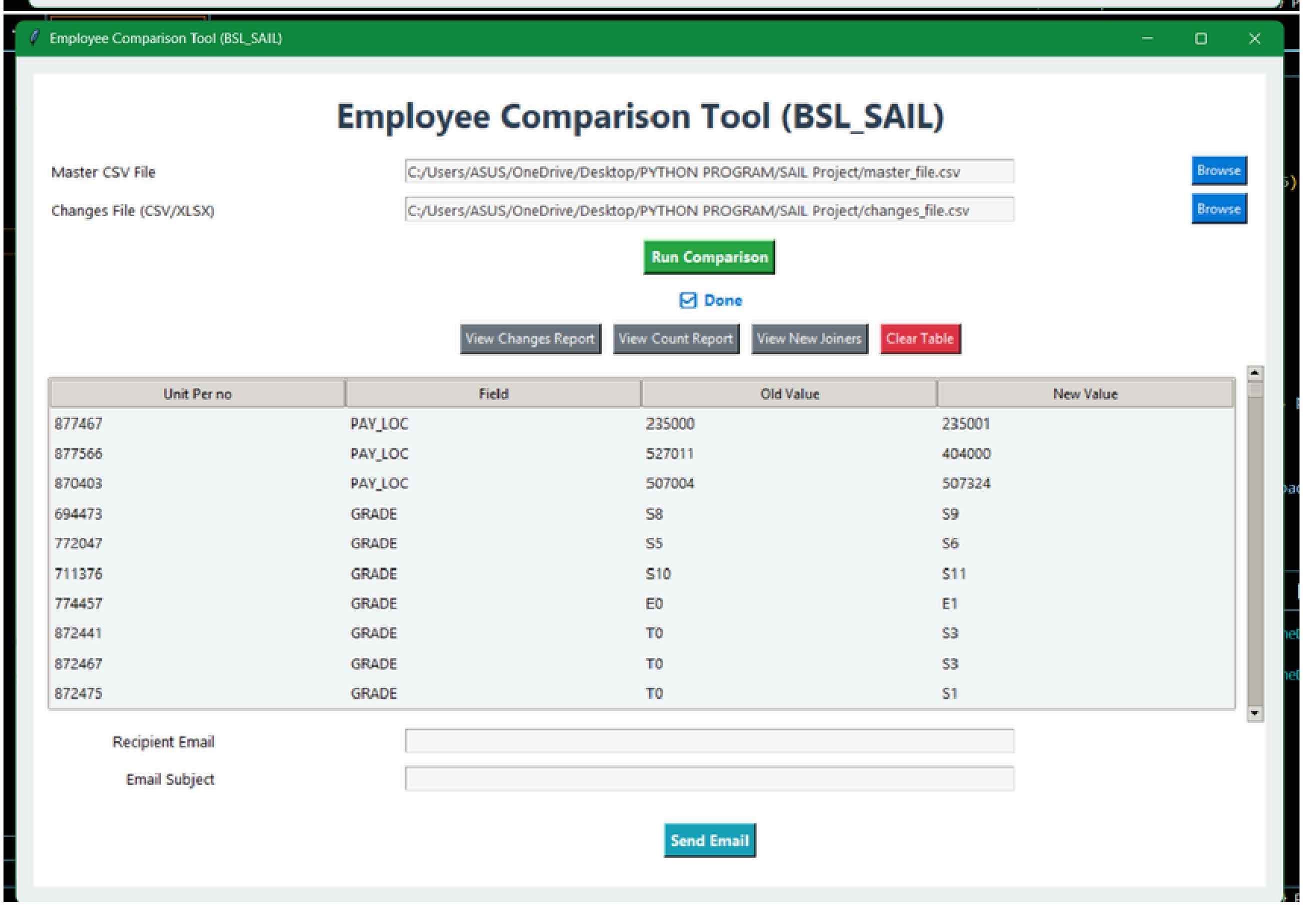
```python
ProjectSAIL.py > ...
def run_comparison():
        update_progress("75% - Extracting new joinees...")
        new_joiners = changes_df[~changes_df["UNIT_PERNO"].isin(master_df["UNIT_PERNO"])]
        fields = ["UNIT_PERNO", "SAIL_PERNO", "PAN", "IFSC_CD", "BANK_ACNO", "UNIT_JOIN_DT", "DOJ_SAIL"]
        new_joiners = new_joiners[[col for col in fields if col in new_joiners.columns]]
        new_joiners.to_csv("New_Joinees.csv", index=False)

        update_progress("90% - Saving reports...")
        count_report = df_changes.groupby("Field").size().reset_index(name="Count")
        count_report = pd.concat([
            count_report,
            pd.DataFrame([{"Field": "new_joinees", "Count": new_joiners.shape[0]}])
        ], ignore_index=True)
        count_report.to_csv("Count.csv", index=False)

        display_data(df_changes)
        update_progress("✅ Done")
        messagebox.showinfo("Success", "Comparison complete. Reports generated.")
    except Exception as e:
        messagebox.showerror("Error", str(e))

def display_data(df):
    tree.delete(*tree.get_children())
    tree.configure(columns=list(df.columns))
    for col in df.columns:
        tree.heading(col, text=col)
        tree.column(col, width=120)
    for _, row in df.iterrows():
        tree.insert("", "end", values=list(row))

def load_report(filename):
    if not os.path.exists(filename):
        messagebox.showerror("Error", f"{filename} not found.")
        return
    df = pd.read_csv(filename)
    display_data(df)
def clear_table():
    tree.delete(*tree.get_children())

def send_email():
    to_email = to_email_var.get()
    subject = subject_var.get()

    if not to_email or not subject:
        messagebox.showerror("Error", "Recipient email and subject are required.")
        return

    try:
        msg = EmailMessage()
        msg["Subject"] = subject
        msg["From"] = "your_email@gmail.com"
        msg["To"] = to_email
        msg.set_content("Please find the attached reports.")

        for file in ["Changes_New.csv", "New_Joinees.csv", "Count.csv"]:
            if os.path.exists(file):
                with open(file, "rb") as f:
                    msg.add_attachment(f.read(), maintype="application", subtype="octet-stream", filename=file)

        with smtplib.SMTP_SSL("smtp.gmail.com", 465) as smtp:
            smtp.login("aryansinha6205@gmail.com", "potm jqht twsi iqlm")
            smtp.send_message(msg)

        messagebox.showinfo("Success", "Email sent successfully.")

    except Exception as e:
        messagebox.showerror("Email Error", str(e))

root = tk.Tk()
root.title("Employee Comparison Tool (BSL_SAIL)")
root.geometry("1100x800")
apply_styles(root)
```

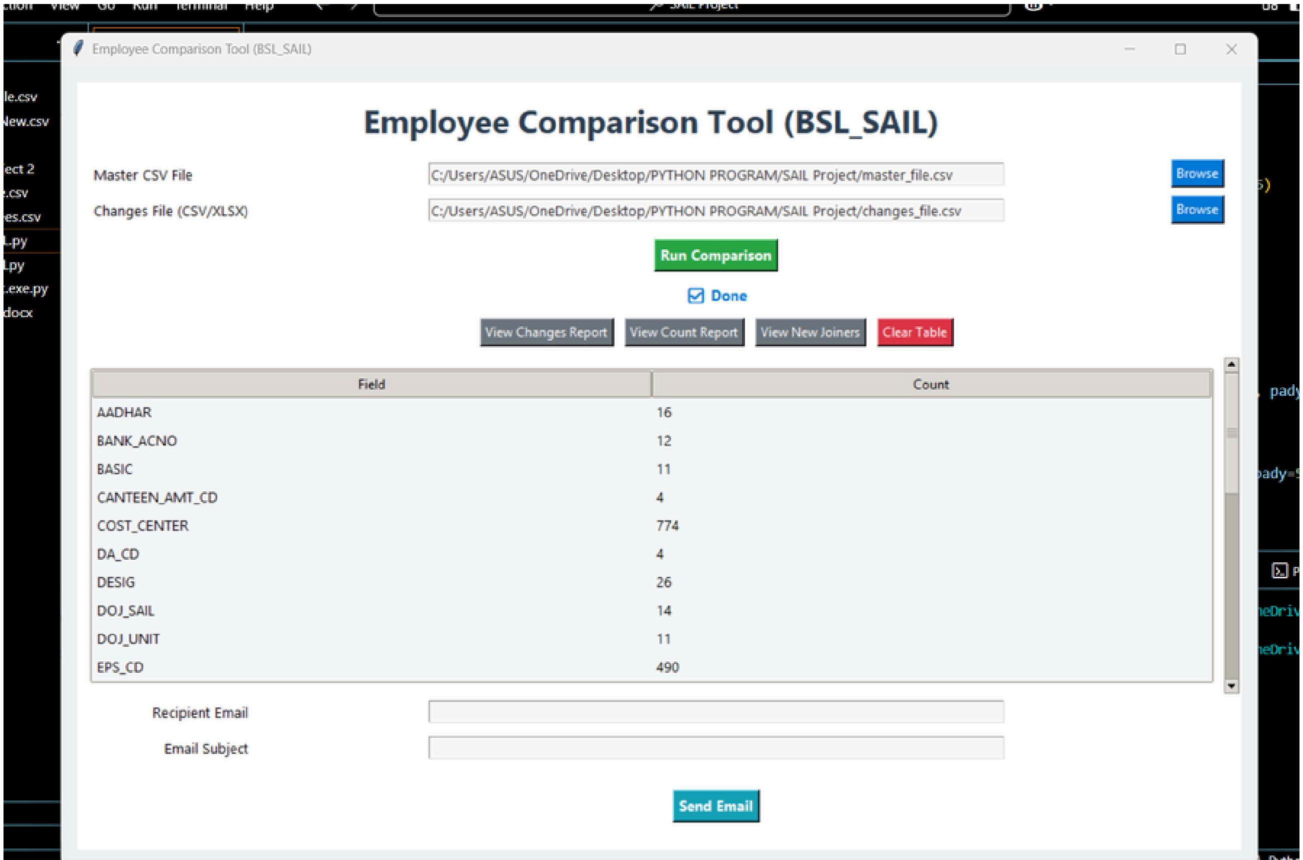
```python
master_file_path = tk.StringVar()
changes_file_path = tk.StringVar()
to_email_var = tk.StringVar()
subject_var = tk.StringVar()

frame = tk.Frame(root, bg="#ffffff", bd=2)
frame.pack(fill=tk.BOTH, expand=True, padx=15, pady=15)

header = tk.Label(frame, text="Employee Comparison Tool (BSL_SAIL)", font=("Segoe UI", 22, "bold"), bg="#ffffff", fg="#2c3e50")
header.grid(row=0, column=0, columnspan=3, pady=10)

entry_opts = {'font': ("Segoe UI", 10), 'bg': "#f9f9f9", 'fg': "#333", 'relief': tk.GROOVE}

row_counter = 1
for label, var, ftype in [("Master CSV File", master_file_path, [("CSV files", "*.csv")]),
                          ("Changes File (CSV/XLSX)", changes_file_path, [("CSV", ".csv"), ("Excel", ".xlsx")])]:
    tk.Label(frame, text=label, font=("Segoe UI", 10), bg="#ffffff").grid(row=row_counter, column=0, sticky="w", padx=10, pady=5)
    tk.Entry(frame, textvariable=var, width=75, **entry_opts).grid(row=row_counter, column=1, padx=5)
    tk.Button(frame, text="Browse", command=lambda v=var, f=ftype: load_file(v, f), bg="#0078D7", fg="white").grid(row=row_counter, column=2)
    row_counter += 1

tk.Button(frame, text="Run Comparison", bg="#28a745", fg="white", font=("Segoe UI", 10, "bold"), command=run_comparison).grid(row=row_counter, column=1, pady=10)
row_counter += 1

progress_label = tk.Label(frame, text="", fg="#0078D7", font=("Segoe UI", 10, "bold"), bg="#ffffff")
progress_label.grid(row=row_counter, column=1)
row_counter += 1

btn_frame = tk.Frame(frame, bg="#ffffff")
btn_frame.grid(row=row_counter, column=1, pady=10)
for i, (text, cmd) in enumerate([
    ("View Changes Report", lambda: load_report("Changes_New.csv")),
    ("View Count Report", lambda: load_report("Count.csv")),
    ("View New Joiners", lambda: load_report("New_Joinees.csv")),
    ("Clear Table", clear_table)]):
    tk.Button(btn_frame, text=text, command=cmd, bg="#6c757d" if i < 3 else "#dc3545", fg="white").grid(row=0, column=i, padx=5)
row_counter += 1

tree = ttk.Treeview(frame, show="headings")
tree.grid(row=row_counter, column=0, columnspan=3, padx=10, pady=10, sticky="nsew")
scrollbar_y = ttk.Scrollbar(frame, orient="vertical", command=tree.yview)
tree.configure(yscrollcommand=scrollbar_y.set)
scrollbar_y.grid(row=row_counter, column=3, sticky='ns')
row_counter += 1

tk.Label(frame, text="Recipient Email", bg="#ffffff", font=("Segoe UI", 10)).grid(row=row_counter, column=0, sticky="e", padx=10, pady=5)
tk.Entry(frame, textvariable=to_email_var, width=75, **entry_opts).grid(row=row_counter, column=1)
row_counter += 1

tk.Label(frame, text="Email Subject", bg="#ffffff", font=("Segoe UI", 10)).grid(row=row_counter, column=0, sticky="e", padx=10, pady=5)
tk.Entry(frame, textvariable=subject_var, width=75, **entry_opts).grid(row=row_counter, column=1)
row_counter += 1

tk.Button(frame, text="Send Email", command=send_email, bg="#17a2b8", fg="white", font=("Segoe UI", 10, "bold")).grid(row=row_counter, column=1, pady=10)

frame.grid_rowconfigure(row_counter, weight=1)
frame.grid_columnconfigure(1, weight=1)

root.mainloop()
```

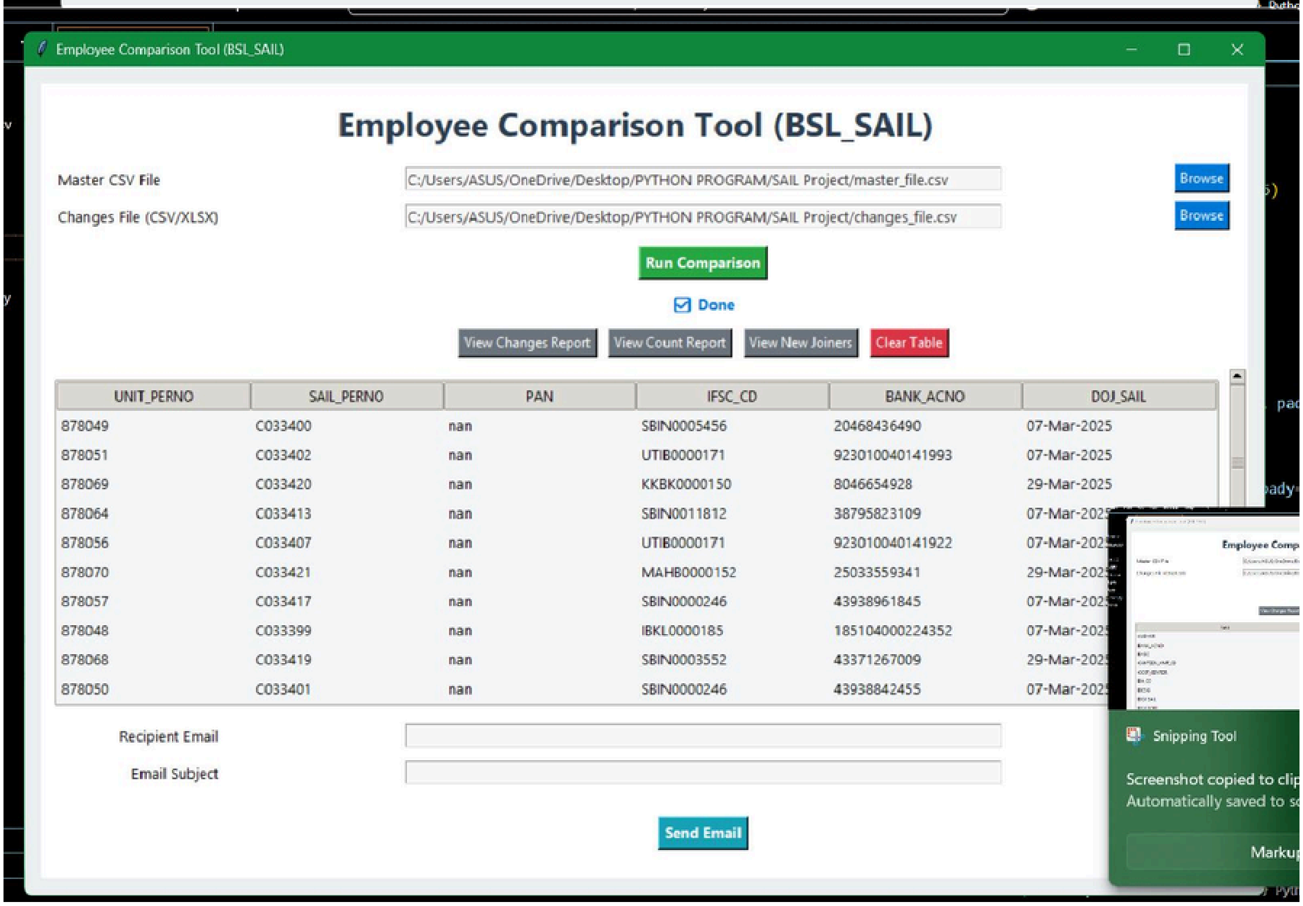
OUTPUT

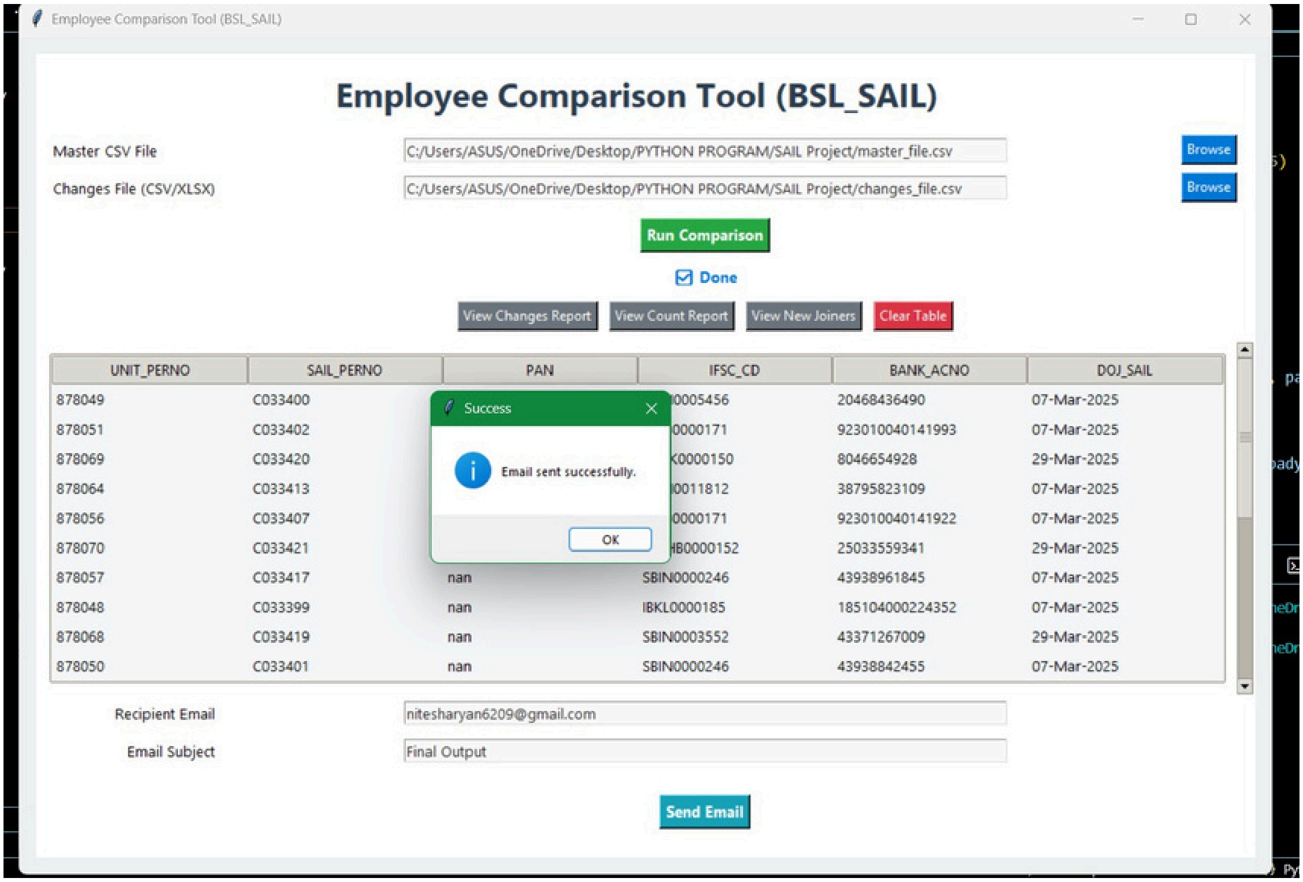

Employee Comparison Tool (BSL_SAIL)
Employee Comparison Tool (BSL_SAIL)
Master CSV File
C:/Users/ASUS/OneDrive/Desktop/PYTHON PROGRAM/SAIL Project/master_file.csv
Browse
Changes File (CSV/XLSX)
C:/Users/ASUS/OneDrive/Desktop/PYTHON PROGRAM/SAIL Project/changes_file.csv
Browse
Run Comparison
Done
View Changes Report
View Count Report
View New Joiners
Clear Table
Unit Per no
Field
New Value
877467 PAY_LOC 235001
877566 PAY_LOC 404000
870403 PAY_LOC 507324
694473 GRADE S9
772047 GRADE S6
711376 GRADE S10 S11
774457 GRADE E0 E1
872441 GRADE T0 S3
872467 GRADE T0 S3
872475 GRADE T0 S1
Success
Comparison complete. Reports generated.
OK
Recipient Email
Email Subject
Send Email


Employee Comparison Tool (BSL_SAIL)
Employee Comparison Tool (BSL_SAIL)
Master CSV File
Browse
Changes File (CSV/XLSX)
Browse
Run Comparison
View Changes Report
View Count Report
View New Joiners
Clear Table
Recipient Email
Email Subject
Send Email


Employee Comparison Tool (BSL_SAIL)
Employee Comparison Tool (BSL_SAIL)
Master CSV File
C:/Users/ASUS/OneDrive/Desktop/PYTHON PROGRAM/SAIL Project/master_file.csv
Browse
Changes File (CSV/XLSX)
C:/Users/ASUS/OneDrive/Desktop/PYTHON PROGRAM/SAIL Project/changes_file.csv
Browse
Run Comparison
Done
View Changes Report
View Count Report
View New Joiners
Clear Table
Unit Per no
Field
Old Value
New Value
877467 PAY_LOC 235000 235001
877566 PAY_LOC 527011 404000
870403 PAY_LOC 507004 507324
694473 GRADE S8 S9
772047 GRADE S5 S6
711376 GRADE S10 S11
774457 GRADE E0 E1
872441 GRADE T0 S3
872467 GRADE T0 S3
872475 GRADE T0 S1
Recipient Email
Email Subject
Send Email


Employee Comparison Tool (BSL_SAIL)
Employee Comparison Tool (BSL_SAIL)
Master CSV File
C:/Users/ASUS/OneDrive/Desktop/PYTHON PROGRAM/SAIL Project/master_file.csv
Browse
Changes File (CSV/XLSX)
C:/Users/ASUS/OneDrive/Desktop/PYTHON PROGRAM/SAIL Project/changes_file.csv
Browse
Run Comparison
Done
View Changes Report
View Count Report
View New Joiners
Clear Table
Field
Count
AADHAR 16
BANK_ACNO 12
BASIC 11
CANTEEN_AMT_CD 4
COST_CENTER 774
DA_CD 4
DESIG 26
DOJ_SAIL 14
DOJ_UNIT 11
EPS_CD 490
Recipient Email
Email Subject
Send Email


Employee Comparison Tool (BSL_SAIL)
Employee Comparison Tool (BSL_SAIL)
Master CSV File
C:/Users/ASUS/OneDrive/Desktop/PYTHON PROGRAM/SAIL Project/master_file.csv
Browse
Changes File (CSV/XLSX)
C:/Users/ASUS/OneDrive/Desktop/PYTHON PROGRAM/SAIL Project/changes_file.csv
Browse
Run Comparison
Done
View Changes Report
View Count Report
View New Joiners
Clear Table
UNIT_PERNO SAIL_PERNO PAN IFSC_CD BANK_ACNO DOJ_SAIL
878049 C033400 nan SBIN0005456 20468436490 07-Mar-2025
878051 C033402 nan UTIB0000171 923010040141993 07-Mar-2025
878069 C033420 nan KKBK0000150 8046654928 29-Mar-2025
878064 C033413 nan SBIN0011812 38795823109
878056 C033407 nan UTIB0000171 923010040141922
878070 C033421 nan MAHB0000152 25033559341
878057 C033417 nan SBIN0000246 43938961845
878048 C033399 nan IBKL0000185 185104000224352
878068 C033419 nan SBIN0003552 43371267009
878050 C033401 nan SBIN0000246 43938842455
Recipient Email
Email Subject
Send Email
Snipping Tool


Employee Comparison Tool (BSL_SAIL)
Employee Comparison Tool (BSL_SAIL)
Master CSV File
C:/Users/ASUS/OneDrive/Desktop/PYTHON PROGRAM/SAIL Project/master_file.csv
Browse
Changes File (CSV/XLSX)
C:/Users/ASUS/OneDrive/Desktop/PYTHON PROGRAM/SAIL Project/changes_file.csv
Browse
Run Comparison
Done
View Changes Report
View Count Report
View New Joiners
Clear Table
UNIT_PERNO SAIL_PERNO PAN IFSC_CD BANK_ACNO DOJ_SAIL
878049 C033400 20468436490 07-Mar-2025
878051 C033402 923010040141993 07-Mar-2025
878069 C033420 8046654928 29-Mar-2025
878064 C033413 38795823109 07-Mar-2025
878056 C033407 923010040141922 07-Mar-2025
878070 C033421 25033559341 29-Mar-2025
878057 C033417 nan SBIN0000246 43938961845 07-Mar-2025
878048 C033399 nan IBKL0000185 185104000224352 07-Mar-2025
878068 C033419 nan SBIN0003552 43371267009 29-Mar-2025
878050 C033401 nan SBIN0000246 43938842455 07-Mar-2025
Success
Email sent successfully.
OK
Recipient Email
nitesharyan6209@gmail.com
Email Subject
Final Output
Send Email

CONCLUSION

The CSV Comparator project successfully demonstrates an efficient and user-friendly tool for identifying and analyzing differences between two data files, typically used for HR or payroll records. By leveraging Python's pandas library and the Tkinter GUI toolkit, the application enables quick comparison based on key columns, highlights discrepancies, and identifies new entries with ease.

The tool not only streamlines the process of comparing large datasets but also ensures that updates, such as changes in employee details or additions of new joiners, are tracked accurately. With features like a readable Excel export, filtering of non-essential columns, and a clear interface, the application adds significant value in real-world administrative and audit processes.

This project highlights the practical application of programming and data handling skills and can be further enhanced by adding features like multi-file support, Data Handling, and database integration.